

Towards an Efficient Containerized Cloud Gaming Platform

Adrien Gegout ^{*} [†], Djob Mvondo [†] [‡], Davide Frey [†] [‡], Pascal Monchon^{*}

^{*}Blacknut, France

[†]Univ Rennes, Inria, France [‡]CNRS, IRISA, France

Abstract—Virtual machines (VMs) raise several challenges for efficiently accommodating Cloud-gaming needs. The first challenge concerns VM boot time. Booting a VM can take several milliseconds to several seconds based on the operating system (OS) and the modules that must be loaded. Consequently, waiting for a VM to boot upon a user's game-start request impacts the game startup latency and, thus, user experience. To mitigate this startup issue, Cloud-gaming providers maintain a pool of pre-initialized VMs, which leads to resource waste when the pool is not fully used. The second challenge concerns the flexibility of VMs regarding their memory size due to the lack of memory hotplug and unplug features. VMs cannot adapt their memory size to accommodate the heterogeneous memory requirements of existing video games. Concretely, pre-initialized VMs for a specific game cannot always be reused to host another game with different memory requirements. This means pre-initialized VMs must be configured with large memory sizes to host several games, leading to memory waste.

In this paper, we bootstrap a discussion to steer away from a VM-centric Cloud-gaming infrastructure in favor of more resource-efficient infrastructures. To this end, we take a two-fold approach. First, we propose using containers as an efficient alternative to Windows VMs for video games. Our experiments show that the lightweight nature of containers achieves up to 10.6% energy-usage reduction without impacting the gaming experience. Second, we propose a novel GPU-sharing idea tailored for game containers. We discuss state-of-the-art GPU-sharing approaches, show their practical limitations for Cloud-gaming platforms, and present our envisioned GPU-sharing approach.

I. INTRODUCTION

Cloud-gaming insights and VMs. The Cloud-gaming market is valued at up to 9.7 billion \$. It is expected to continuously grow [1], [2] as several Cloud-gaming actors continue to innovate, such as Amazon Luna [3], Blacknut [4], Boosteroid [5], Ludium [6], Nvidia Geforce Now [7], This growth stems from the several advantages Cloud gaming provides compared to traditional gaming for end users. Indeed, with Cloud gaming, users are no longer bound to the hardware characteristics of their devices, as all the game processing is performed by a remote server hosted by the Cloud-gaming provider. Concretely, during a game session hosted by the Cloud-gaming computing platform, users send their inputs, which are processed by the remote server, and the game output is streamed back to the user's end device [8]–[11]. Any defect will reduce the user's quality of experience and indirectly undermine the trust from game developers and editors. Thus, the provider has two crucial goals: (1) ensuring good user experience while (2) achieving resource-usage efficiency, which is correlated to the computing costs.

To achieve both goals, providers run each game in sandboxes, which are isolated from each other so that a server can host several sandboxes in parallel to maximize resource usage. The preferred sandbox for games consists of virtual machines (hereafter denoted as VMs). This is due to the peculiar nature of game software. Games heavily rely on user-space graphic libraries (e.g., Vulkan [12], DirectX [13], OpenGL [14], etc.), and are strongly tied to a specific operating system coupled to specific hardware drivers [9]. The ability of VMs to virtualize an entire operating system with its drivers thus makes them a very popular choice for Cloud-gaming platforms [15]–[22]. Cloud-gaming servers typically host VMs running games, and different VMs must compete for the most critical game resource, the graphics processing unit(s) (GPU).

VMs and GPU sharing issues. For Cloud gaming, VMs raise two main issues affecting host servers' resource efficiency. The first concerns the boot time of VMs, which can take from several milliseconds to even seconds to initialize correctly because they require booting a full operating system [23]–[26]. Additionally, game binaries must be attached to VMs as virtual disks, adding even more delay. Consequently, an on-demand initialization mechanism for launching games will incur prohibitive latency (Figure 1) for the game to launch, negatively affecting user experience. Moreover, responding to such latency spikes by spawning new VMs would not help, as their boot time quickly becomes a bottleneck.

A solution used by Cloud-gaming providers consists of maintaining a pool of initialized VMs where games can be launched directly without paying the initialization cost. However, this wastes resources when the pool is unused, as the initialized VMs use disk and memory space, and their idle OS monopolizes a fraction of the computing power.

The second issue concerns the flexibility of VMs. Indeed, some VMs' resources cannot be changed at runtime, such as their associated CPU cores and memory allocation [27], [28] or GPU hard partitioning (e.g., Nvidia vGPUs [29]). The direct consequence is that a VM sized for a specific game cannot be reused to host a game with different resource requirements. Thus, Cloud gaming providers that maintain a pool of VMs must configure large VMs with resources that can host several games, as games have heterogeneous resource (memory, GPU, etc.) requirements. This situation ultimately leads to resource waste, as the resources provisioned by the VMs won't always be fully used.

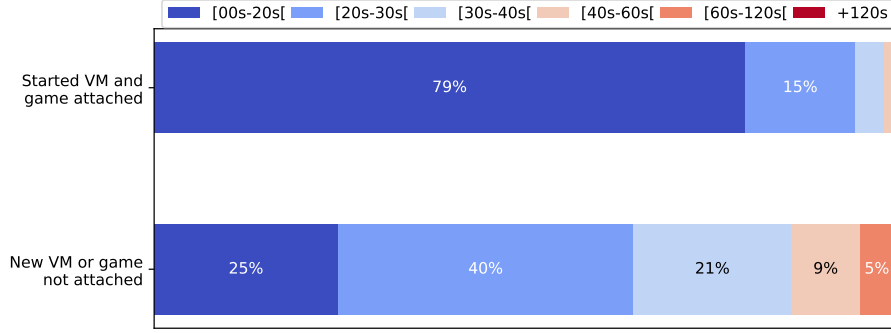


Fig. 1: Proportion of cloud-gaming sessions user wait latency in intervals of seconds, based on two years of collected production data containing metrics on more than 2,500,000 successful sessions. “Started VM and game attached” represents the optimal case for minimum user wait delay, where the VM is already booted, and the target game volume is already attached. “New VM or game not attached” represents the non-optimal case, where either the VM is booted at user demand or the target game volume is not already mounted to the selected existing VM.

Furthermore, sharing the GPU among the different VMs is difficult. Games cannot be preempted and rescheduled easily on GPUs like other workloads, such as machine-learning (ML) training tasks, due to how games store textures and other objects in the GPU memory [21]. This leads to solutions such as dedicating the GPU to one VM, which limits the server’s capacity to host other VMs, or performing hard partitioning of the GPU, which is not a flexible approach for the heterogeneous demands of video-game applications.

At first sight, the problems associated with the use of VMs for cloud gaming may seem solvable at least partially through the use of Unikernels [30]–[33]. Unikernels are specialized OS built for specific target applications, eliminating general-purpose unnecessary features. This makes unikernels much faster than VMs and sometimes even faster than containers [34]. However, Unikernels remain ill-suited for Cloud gaming for several reasons. First, cloud gaming providers rarely, if ever, have access to the source code of the games they propose. Second, GPUs are generally not supported for rendering purposes in unikernels. Existing solutions [35], [36] only support GPU computation but not the extensive graphical rendering features required by games. Third, available unikernels include stripped-down versions of core libraries [31], which fail to satisfy the complex dependencies of most games. Moreover, these dependencies vary from game to game. As a result, applying a unikernel approach would require in-depth customization of the unikernel for each specific game to work. **Our work.** Due to the inapplicability of unikernels, Containers appear to constitute the most promising alternative to VMs concerning Cloud-gaming requirements. We know that containers are generally more efficient than VMs [37] and even for Linux native gaming [38]. The question remains whether they can compete with VMs regarding performance and resource consumption in the context of most modern gaming workloads, including Windows’ ones.

In this paper, we answer these questions by presenting the results of extensive experiments on four video games in the context of our cloud-gaming platform. Our results show that

not only containers can rival the performance of Windows VMs for gaming, but they can also provide significant energy savings. In particular, containers achieve up to 10.6% less energy usage than Windows VMs without impacting the game performance for four Windows video games: *I’m fish*, *Deliver us the moon*, *Kill it with fire* and *HOT WHEELS UNLEASHED™ 2 - Turbocharged* with different parameters.

Secondly, we examine existing works that target GPU sharing between different tasks. We uncover that existing works are not easily applicable to gaming containers. Additionally, some existing works require modifying the execution logic of the target tasks, which is not always possible for video games as Cloud-gaming providers deal with game binaries. Others require the interception of proprietary API commands, like Microsoft DirectX, which are generally difficult to maintain. Based on our analysis, we present our idea of a GPU-sharing algorithm tailored for video games, which would leverage Linux container specificities.

Contributions. To summarize, this paper makes the following contributions:

- an evaluation of containers’ efficiency in a gaming context (game performance and energy usage) compared to Windows VMs that shows up to 10.6% energy reduction without degrading game performance;
- an analysis of the practicality of existing GPU-sharing solutions for the Cloud gaming context;
- and an early discussion of a GPU-sharing algorithm, considering that games run in containers.

II. CONTAINERS FOR CLOUD GAMING

As mentioned previously, VMs suffer from boot latency and flexibility issues that lead to resource-inefficient workarounds, such as maintaining a pool of pre-initialized VMs. In this section, we discuss how containers can provide an effective alternative, as they constitute lightweight isolation environments compared to VMs.

Containers are lightweight sandboxes that isolate processes from each other while enabling fine-grained resource throttling and monitoring. Containers on the same host share the same kernel as the host, while each VM can have its own full OS. Consequently, containers can boot considerably faster than VMs since the OS and required base libraries are already loaded. Several efforts from the GPU community have made containers able to interact with host GPUs (mainly NVIDIA GPUs [39]). Thus, games can be run on containers and benefit from their flexibility and fast-boot potential. Furthermore, if the host OS possesses all the libraries required by the game, it is unnecessary to modify the game's source code or recompile it to launch it within a container. In this section, we distinguish two types of containers: Windows containers and Linux containers.

Lastly, because of their fast deployment from their base image, containers can be made for single use without impacting quality of experience (QoE). This is the opposite of VMs, which should be reused between user sessions to avoid generating delays. These ephemeral containers prevent accidental reuse of potentially sensitive user data and traces, as every container and the data they contain is entirely deleted at the end of its associated user session.

Windows containers. Up to 60% of the games registered on the Steam¹ store (a mainstream popular video-game store) are only compatible with the Windows OS [40], [41]. This is mainly due to their video game dependence' on the DirectX API, which includes the Direct3D graphical API, a Microsoft proprietary API used by Windows-native applications to interact with GPUs. Such games rely on a version of DirectX to render their audio and video. We assumed that containers running atop the Windows OS could directly benefit from these APIs without requiring additional support.

Linux containers. As highlighted previously, most games are only compatible with the Windows OS and, as such, are not natively compatible with Linux systems. Hindering the adoption of containers for video games on Linux platforms.

However, there exist open-source components specialized for running Windows-based processes in a Linux environment, such as Wine [42]–[44]. Wine is used as a base by the Valve Proton project [45], [46], specialized in running Windows games on Linux systems. Consequently, games can also theoretically be launched within Linux containers and benefit from the advantages of containers regarding resource usage and flexibility.

¹<https://store.steampowered.com/>

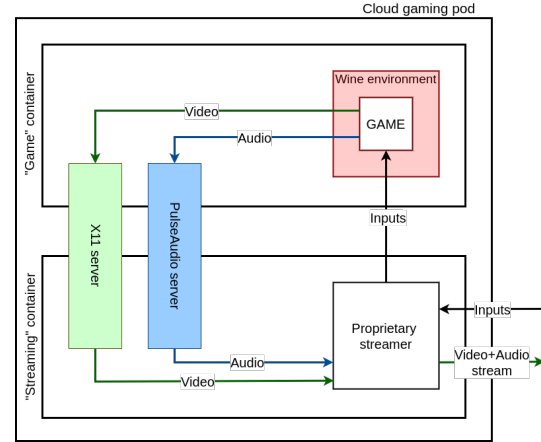


Fig. 2: Containerized Cloud-gaming architecture. Each game instance consists of two containers: the “game” container, where the game runs, and the “streaming” container, which streams the video and audio output to the network.

Overall, containers can fully isolate video games, providing better resource management flexibility. To ensure of their viability with video games’ workload, we investigate their performance and compare them to VMs.

III. CONTAINERS AGAINST VMs

We evaluate video game performance in containers and VM environments and collect server resource usage to assess containers’ potential gains and overhead.

Container gaming architecture details. We consider a Linux VM and a Windows VM for our VMs configurations.

Regarding containers, despite our efforts, running any graphical process inside a Windows container was impossible as they lack a window state [47] and cannot render any graphical object. This issue is well known and is a deliberate design choice from Microsoft [48]. Consequently, we focused on Linux containers to assess the potential of using containers for Cloud-gaming.

Our containerized Cloud-gaming environment runs one game per instance. Several instances can be executed on the same operating system at the same time. As shown in Figure 2, each instance consists of two Linux containers, the *game* container (similar to [38]) and the *streaming* container.

We use Docker [49] to manage our containers’ life cycles. The two containers share some of their isolation layers. Specifically, they share their PID and IPC layers, allowing the processes they run to interact easily. For efficiency, the *streaming* container has no network isolation, directly using the host network interfaces, as any classic process would do.

In a production environment, the *streaming* container should only contain processes trusted by the cloud-gaming provider (e.g. streaming process), while the *game* container can contain untrusted processes. Specifically, games can be considered blackboxes with unknown intents.

Even if we do not focus on security aspects in this paper, this two-container architecture provides a basic form of protection by isolating (i) trusted components from game processes and (ii) the game’s potentially sensitive user data from the Cloud-gaming provider’s tools, thus preventing accidental data collection. Games from different containerized instances running in the same system are completely isolated from each other. Moreover, as we instantiate a new containerized environment for each user session, any user data and traces left inside at the end are automatically and irremediably deleted.

In our *game* container, we install all libraries and software needed to adequately run a Windows game inside a Linux environment: (1) “*wine-GE-custom*” [42] of the open-source Wine project [43], [44] to simulate Windows system API calls inside a Linux system, (2) “*DXVK*” [50] and “*DXVK NvAPI*” [51], to translate Microsoft proprietary Direct3D9, 10 and 11 API calls into Vulkan API calls (DXVK NvAPI enables better integration with Nvidia GPUs), (3) “*Vkd3d*” [52], [53] to translate Microsoft proprietary Direct3D 12 API calls into Vulkan API calls, and (4) *MangoHud* [54] for debugging and for collecting game-layer telemetry.

The *streaming* container embeds all the libraries and software needed to run the Cloud-gaming provider’s streaming components (e.g. Gstreamer [55] dependencies) and to build a minimal graphical interface for games, including an X11 or Wayland graphical server bound to the GPU driver and a PulseAudio server. It allows each game to use their own media servers, reinforcing their isolation with each others. The *streaming* container also manages the injection of user inputs inside the game, using udev and cgroups. The two containers also share a file-system folder, allowing them to communicate using Unix sockets.

Testbed. We perform our experiments on a server with an AMD EPYC 7502P 32-core CPU, an Nvidia T4 GPU, and 256 GiB of RAM. The container and VM instances are configured with a limited cpuset of 8 vCPU (4 physical cores) each and limited to 10 GiB of RAM each. The server host runs Ubuntu 22.04 and the containers are built on top of the official Ubuntu 22.04 docker version. The Windows VMs run Windows 2019, and the Linux VMs run Ubuntu 22.04. The two VMs use the Nvidia vGPU guest driver.

To ensure comparable results, the Linux VM environment is a mirror of the Linux containers environment’s configuration, minus the additional isolation between streaming components.

Game and experimental parameters. We run each game in a Linux container, as well as in Windows and Linux VMs. We consider the four following games:

- “*I am fish*”, Mars 2021, developed by **Bossa studios** and edited by **Curve Games**.
- “*Deliver us the moon*”, September 2018, developed by **KeokeN Interactive** and edited by **KeokeN Interactive** and **Wired Productions**. The game can run on DirectX11 or DirectX12, depending on a command-line argument.
- “*Kill it with fire*”, August 2020, developed by **KeokeN Interactive** by **Casey Donnellan Games LLC** and published by **tinyBuild**. This game uses DirectX11.

- “*Hot Wheels Unleashed 2: Turbocharged*”, October 2023, developed and published by **Milestone S.r.l.**. This game uses DirectX12.

These games were selected because of their popularity within a cloud gaming provider.

We evaluated *I am fish* in 60 FPS and 30 FPS with its “high” graphic quality preset. *Deliver us the moon* was evaluated with DirectX11 and with DirectX12 APIs, both at 60 FPS and both with its “epic” preset. *Kill it with fire* was evaluated in 60 FPS with its “epic” preset. *Hot Wheels Unleashed 2: Turbocharged* was evaluated in 30 FPS with its “low” preset to ensure a comparable result with Linux VMs.

Due to a compatibility issue between the Vulkan graphic API and our version (550.54.10) of the Nvidia vGPU driver, it was impossible to test *Deliver us the moon* with the DirectX12 backend on a Linux VM. Each experiment is performed two times for five minutes each.

Metrics and experimental methodology. We are interested in measuring the host server resource usage during a gaming session and the game performance (FPS) for both sandboxing solutions: VM and containers.

Concretely, we collect :

- CPU-usage counters from */proc/stat*.
- Memory-usage metrics from */proc/meminfo*.
- Disk-usage metrics and counters from */proc/diskstats*.
- AMD EPYC Model-specific registers (MSR) to collect RAPL power-usage counters for each physical core and the global CPU package, from */dev/cpu/*/msr*.
- GPU-usage metrics and counters from the NVIDIA Management Library (NVML).

Each of these metrics is collected once per second. Since each run lasts 300 seconds, there are 300 records of each metric per run. During the tests, we ensure with *MangoHud* [54] (Linux containers and Linux VMs) and *PresentMon* [56] (Windows VMs) that games are running smoothly at the target frames per second: 30 or 60 FPS (Figure 5). Due to an incompatibility with the Windows VM environment, *PresentMon* was unable to save FPS metrics to disk, even if it still allowed to visually verify FPS and associated framerate of the tested games. Each test session consists of five minutes of simple character movements as depicted in Figure 3 in a specific game level for the first three games.

For *Hot Wheels Unleashed 2: Turbocharged*, we leveraged the game’s AI to automatically simulate a real user. For *I am fish*, we selected the tutorial level and made the fish character jump in and out of the water. For *Deliver us the moon*, we selected the first level and made lateral movements. For *Kill it with fire*, we started a new game and made back-and-forth movements in a corridor.

In each game, the movements were made to force at least a moderate GPU and CPU activity while maintaining stable resource usage, ensuring good comparability between results.



Fig. 3: In-game movements (green arrows) made during the tests. Two screenshots of the first level of the games 'I am fish' and 'Deliver us the moon', each with an overlay arrow displaying the game's character movements performed during the tests.

In the following sections, we will present our experimental results. Overall, there is no major drop in FPS for both settings, i.e., container and VMs (Figure 5). This highlights their performance similarity regarding user experience and, consequently, the adequacy of containers for running games.

A. CPU usage comparison

As VMs and containers have different impacts on CPU system, guest, and user time, we display our result in terms of CPU usage, inferred from the collected CPU idle time. Figure 4a reports the results regarding the CPU usage for each configuration. We observe that the Linux container environment uses the least CPU time of all configurations. This result stems from the container having fewer components than a full-fledged OS that runs in a VM, even with a graphical API translation layer. The Windows VM follows the Linux container, while the Linux VM performs the worst with the highest CPU usage across all evaluated games. Isolating the exact cause of why the Linux VM uses more CPU resources than the Windows VM is hard as there are several tightly coupled components and determining the impact of each without noise is not straightforward. However, we suspect this observation is related to the Nvidia Linux vGPU guest driver lower compatibility with the Vulkan API, hypothetically putting a strain on the system.

Observation 1: Overall, across all games, containers exhibit less CPU usage than VMs. Linux VMs use the most CPU time.

B. GPU usage comparison

Using NVIDIA NVML ("*nvidia-smi*" CLI tool [57]), Nvidia describes the GPU usage metrics as the "Percent of time over the past sample period during which one or more kernels was executing on the GPU"². In our experiments, the sample period is of one second. Figure 4b reports GPU streaming multiprocessing (SM) usage results for all configurations.

It shows no significant difference between the container and the Windows VM regarding GPU SM usage. We can still see that containers use slightly more GPU SM processing time for some games.

²Using the "*nvidia-smi --help-query-gpu*" command

Observation 2: Overall, across all games, all settings are generally on par regarding the resource usage of GPU SM. Thus, containers only introduce a negligible GPU overhead for some games.

C. Energy consumption comparison

We use existing probes included in CPUs and GPUs to measure the energy consumed by each game configuration in each game environment. Concretely, we leverage RAPL MSR and NVML to estimate the energy used for each configuration. Figure 6, the "CPU" is the summed energy consumption of all the AMD EPYC 7502P 32 physical cores plus uncore hardware components, as reported in their MSR [58].

For each experiment, we run the container and VM twice, i.e., launching a game, exiting it (first launch), and then relaunching it (second launch). We record the energy consumption for each test.

The DRAM energy consumption is not measured for this specific CPU model [59] and does not appear in our results. Figure 6' is the global energy consumption of the Nvidia T4 GPU, as reported by NVML. It should be noted that the maximum energy draw of the Nvidia Tesla T4 GPU reported by Nvidia is 70 W.

With Figure 6, we observe that, in general, containers are the most energy-efficient game environment, with the lowest reported energy consumption from the AMD EPYC CPU and a somewhat equivalent reported energy consumption from the Nvidia T4 GPU in all three game configurations. Overall, containers improve energy consumption by up to 10.6% with respect to the traditional Windows VM.

Observation 3: Overall, apart from Deliver us to the moon with DirectX12 where results containers are on par with window VMs, containers are the most energy efficient by up to 10.6%. The Linux VM reports the most energy usage.

D. Summary

Our experiments show that containers can replace Windows VMs without affecting game performance (Figure 5), reducing CPU usage and leading up to a 10.6% reduction in energy usage. Our findings correlate with previous CPU only experiments [37] and with Linux only gaming experiments [38]. It can enable cloud gaming providers to benefit from fast startup and flexible resource management of containers, even with Windows games. In the next section, we discuss the second issue towards achieving more efficient cloud gaming platforms: efficient GPU sharing for video games.

IV. SHORTCOMINGS OF EXISTING GPU SHARING APPROACHES FOR CLOUD GAMING

Today, Linux-container isolation enables efficient resource partition and limitation for each container, e.g. CPU cores and quotas, memory, storage access and bandwidth ..., but not for GPU resources. Several containers can use one GPU at the same time, but they cannot easily be limited in their usage.

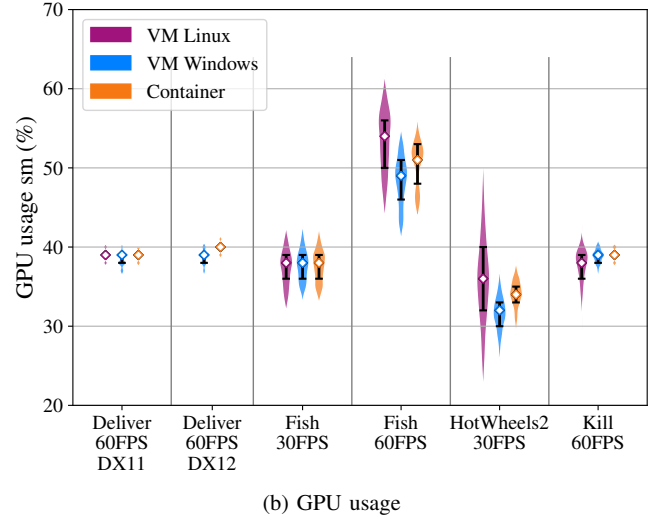
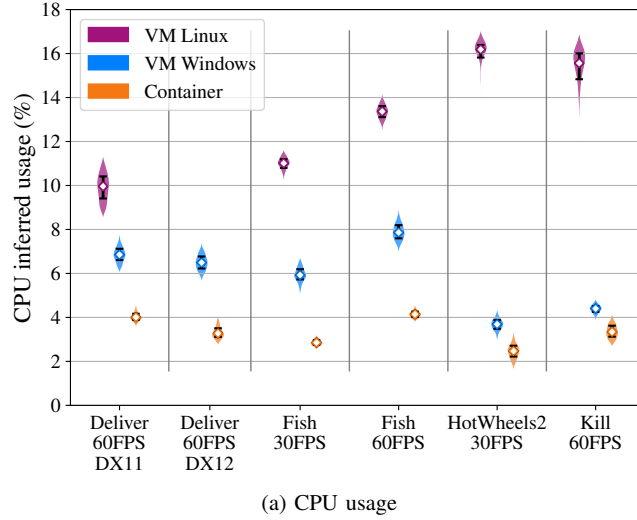


Fig. 4: Violins showing CPU usage time (inferred from CPU idle time) in percent and GPU usage in percent. Violins contain whiskers for Q1, median and Q3.

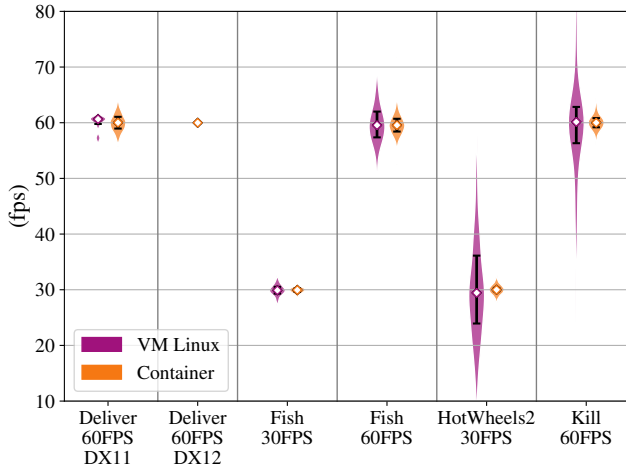


Fig. 5: Violins showing games' FPS per environment. Violins contain whiskers for Q1, median and Q3.

To share GPU between different VMs, one can use proprietary solution from GPU vendors, like Nvidia vGPU or MIG (Multi-Instances GPU). But containers cannot easily benefit from these solutions, and their static partitioning is ill adapted to video games' dynamic workloads [60]. To create a container-based cloud-gaming platform, containers must be able to fairly and efficiently share GPU resources between them. We list here different existing works and their applicability to gaming containers.

Significant work has already addressed the GPU multi-tenancy problem. “gQoS” [61] focused on full virtualization of the GPU, enabling it to be shared among several VMs at the cost of rewriting part of the kernel-side GPU driver. Other works focused on GPU para-virtualization [20], [62], where a subset of GPU capabilities is shared between several VMs after a partial modification of the host GPU driver.

Similarly, machine-learning cloud-computing efficiency has received significant attention lately, with industrial and public adoption of generative models. Although these workloads do not typically require graphical rendering, training or inferring ML models is more efficient on GPUs, opening new research axes on GPU multi-tenancy. “TGS” [63] demonstrated efficient GPU multi-tenancy for ML workloads within Linux containers. Still, the majority of ML-related research cannot be directly applied to cloud gaming, as the subset of shared GPU capabilities does not include graphical rendering [63]–[65] and is generally supported by the Nvidia GPU compute API, CUDA. Since the beginning of 2024, Nvidia Corporation legally locked [66], [67] CUDA API usage to Nvidia GPUs only, limiting CUDA usage.

In other studies [21], [22], [68], the graphical API calls of the target software are intercepted and funneled to another VM that houses GPU(s). This concept is called *API remoting* [69]. “VGRIS” [22] and “ShareRender” [21] intercept a subset of Microsoft proprietary Direct3D calls. “VAR” [68] intercepts Vulkan API calls. This technique can separate the game system from the GPU system and allow fine GPU usage control. However, it comes at the cost of more significant hardware-access latency and the need to partially modify the behavior of the target API.

Nonetheless, modifying closed-source system APIs and drivers, as suggested in gQoS [61] is difficult, costly, and complex to maintain in time. Similarly, modifying the source code of video games as proposed by dJay [9] is not always possible, as cloud-gaming providers rarely have access to it.

To conclude, video games need graphical GPU access, and their GPU multi-tenancy cannot be supported by existing GPU computing-only frameworks (e.g., Nvidia CUDA) as they lack graphical rendering support.

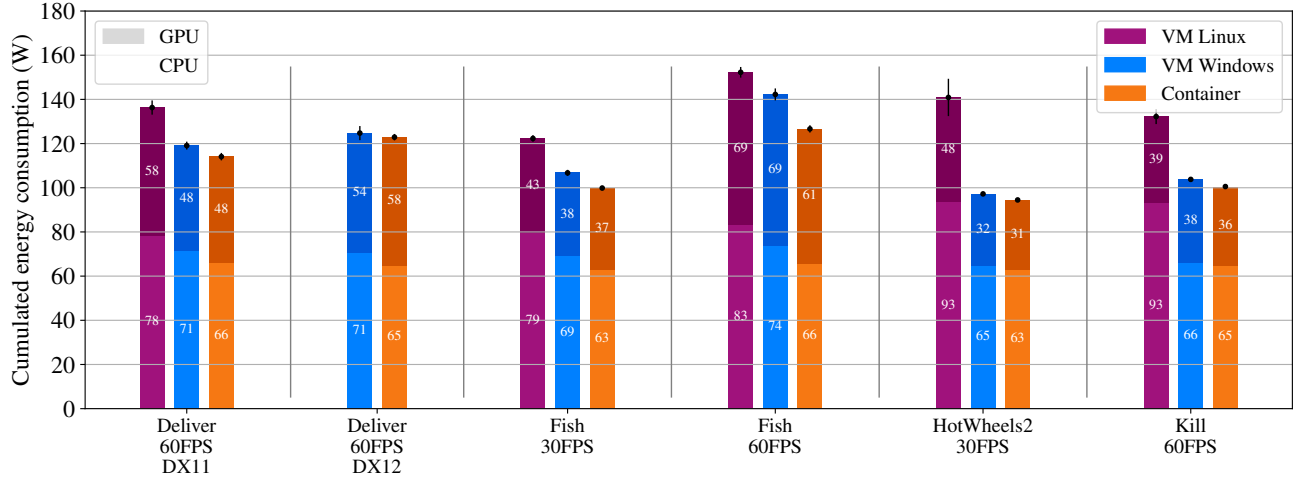


Fig. 6: Cumulated average energy consumption (Watt) of the server CPU and GPU. Error bars represent CPU+GPU standard deviation.

V. FUTURE WORK

We aim to improve the resource efficiency of the existing Cloud-gaming infrastructure. In this respect, our first step was to explore and evaluate alternatives to VMs. We showed that containers can be a solid alternative to Windows VMs that can save resources and energy usage. We intend to consolidate these early results by evaluating containers with a broader range of games to confirm our observations on a larger scale.

In Section IV, we outlined the shortcomings of existing GPU-sharing solutions regarding their adoption in the Cloud gaming context. Our short-term plan is to leverage native Vulkan API interception to reduce delays related to GPU sharing. Concretely, our objective is twofold: 1) We are building a low-latency vendor-agnostic Vulkan graphical API interception (also known as “API remoting”, but in our case we do not modify the API commands’ initial target) component that lets us finely and dynamically control GPU graphical resource usage for each game. 2) We aim to profile several video games and frame-rendering patterns to build a model to predict upcoming GPU usage resulting from near-future game rendering. Using the above principle, we envision building a game-tailored GPU scheduler that enables GPU sharing while meeting video games’ stringent latency and quality demands.

Our current development involves designing a novel GPU-sharing mechanism for co-located containerized video games, leveraging our Linux-based container architecture. As we use *dxvk* and *Vkd3d* graphical API translation layers, games running in our containerized environment use the Vulkan [12] API to use GPU resources. We are using the Vulkan logical component named *Vulkan Loader* [70], which allows us to natively intercept any Vulkan API functions, injecting custom logic into them with minimal overhead. Vulkan API interception enables control over a game’s access to the GPU and underlying resources. It will run transparently without modifying the game’s source code or the Vulkan API.

Our prototype can already limit the quantity of GPU memory (VRAM) used by each game. To this end, it simulates a lower visible amount of reported free GPU memory and returns *out of memory errors* when the game, with the intermediary of *dxvk* or *Vkd3d*, reaches the authorized limit and tries to allocate more GPU memory. Concretely, the prototype dynamically partitions the GPU memory among games.

Contrary to VMs, all Linux containers that run on the same OS share the same logical devices and kernel-space drivers. We leverage this by sharing references to Vulkan GPU synchronization objects between all games on the same system. By controlling these objects, we can control the GPU access of these games with minimal CPU involvement, as the Vulkan GPU synchronization objects are stored within the GPU memory. We expect our first prototype of this feature will enable dynamically controlled GPU temporal sharing. Later, we plan to add spatial GPU sharing as well, as long as the underlying GPU command processor supports it. In this project, we do not plan to modify games’ graphical commands to maintain adequate compatibility with most games.

VI. CONCLUSION

In this paper, we described the first milestone of our work: a working containerized cloud-gaming environment capable of hosting a video game and the associated cloud-gaming components. We showed consistent results with [38] that containers constitute a valid alternative to the current VM-based model, enabling fine-grained resource attribution. Specifically, our experiments showed that our solution is at least as efficient as using traditional Windows VMs for the four tested games on the considered hardware.

We also highlighted the limits of existing GPU-sharing solutions and outlined our future plans for overcoming these limits. After proposing novel gaming-oriented GPU-resource-sharing solutions, we plan to deploy them in a Cloud-gaming environment.

This will allow the cloud gaming infrastructure to support multiple game sessions per server, while allocating only the resources they need, maximizing resource usage and reducing energy consumption.

REFERENCES

- [1] F. B. Insights, "Cloud gaming market size, share & industry analysis, by device (smartphone, laptop/tablets, personal computer, smart tv, and consoles), by streaming type (video streaming and file streaming), by end-user (casual gamers, avid gamers, and hardcore gamers), and regional forecast, 2024-2032," 2024, <https://www.fortunebusinessinsights.com/cloud-gaming-market-102495>.
- [2] Statista, "Cloud gaming - worldwide," 2023, <https://www.statista.com/outlook/amo/media/games/cloud-gaming/worldwide>.
- [3] Amazon, "Luna cloud gaming," <https://luna.amazon.fr/>.
- [4] Blacknut, "Unlimited access to more than 500 games on all your screens," <https://www.blacknut.com/>.
- [5] Boosteroid, "Cloud gaming platform," <https://boosteroid.com/>.
- [6] L. Lab, "Most optimized cloud gaming service," https://www.ludiumlab.com/solutions/cloud_gaming.
- [7] NVIDIA, "Nvidia geforce now," <https://www.nvidia.com/en-us/geforce-now/>.
- [8] J. Meng, S. Paul, and Y. C. Hu, "Coterie: Exploiting frame similarity to enable high-quality multiplayer vr on commodity mobile devices," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 923–937. [Online]. Available: <https://doi.org/10.1145/3373376.3378516>
- [9] S. Grizan, D. Chu, A. Wolman, and R. Wattenhofer, "dJay: enabling high-density multi-tenancy for cloud gaming servers with dynamic cost-benefit GPU load balancing," in *Proceedings of the Sixth ACM Symposium on Cloud Computing*, ser. SoCC '15. Association for Computing Machinery, pp. 58–70. [Online]. Available: <https://dl.acm.org/doi/10.1145/2806777.2806942>
- [10] L. Lin, X. Liao, G. Tan, H. Jin, X. Yang, W. Zhang, and B. Li, "Liverender: A cloud gaming system based on compressed graphics streaming," in *Proceedings of the 22nd ACM International Conference on Multimedia*, ser. MM '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 347–356. [Online]. Available: <https://doi.org/10.1145/2647868.2654943>
- [11] R. D. Yates, M. Tavan, Y. Hu, and D. Raychaudhuri, "Timely cloud gaming," in *IEEE INFOCOM 2017 - IEEE Conference on Computer Communications*, 2017, pp. 1–9.
- [12] Vulkan, "Vulkan," <https://www.vulkan.org/>.
- [13] W. A. Development, "Directx installation for game developers," <https://learn.microsoft.com/en-us/windows/win32/dxtechcharts/directx-setup-for-game-developers#directx-runtime>.
- [14] OpenGL, "Opengl," <https://www.opengl.org/>.
- [15] Y. Han, D. Guo, W. Cai, X. Wang, and V. Leung, "Virtual machine placement optimization in mobile cloud gaming through QoE-oriented resource competition," pp. 1–1. [Online]. Available: <https://ieeexplore.ieee.org/document/9115841/>
- [16] A. Wibowo and T. N. Binh Duong, "CloudNPlay: Resource optimization for a cloud-native gaming system," in *2021 IEEE 30th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*. IEEE, pp. 33–38. [Online]. Available: <https://ieeexplore.ieee.org/document/9680542/>
- [17] Y. Li, Y. Deng, X. Tang, W. Cai, X. Liu, and G. Wang, "On server provisioning for cloud gaming," in *Proceedings of the 25th ACM international conference on Multimedia*. ACM, pp. 492–500. [Online]. Available: <https://dl.acm.org/doi/10.1145/3123266.3123310>
- [18] A. Bhojan, S. P. Ng, J. Ng, and W. T. Ooi, "CloudyGame: Enabling cloud gaming on the edge with dynamic asset streaming and shared game instances," vol. 79, no. 43, pp. 32 503–32 523. [Online]. Available: <https://doi.org/10.1007/s11042-020-09612-z>
- [19] H. Chen, X. Zhang, Y. Xu, J. Ren, J. Fan, Z. Ma, and W. Zhang, "T-gaming: A cost-efficient cloud gaming system at scale," vol. 30, no. 12, pp. 2849–2865, conference Name: IEEE Transactions on Parallel and Distributed Systems.
- [20] C. Zhang, J. Yao, Z. Qi, M. Yu, and H. Guan, "vGASA: Adaptive scheduling algorithm of virtualized GPU resource in cloud gaming," vol. 25, no. 11, pp. 3036–3045.
- [21] W. Zhang, X. Liao, P. Li, H. Jin, and L. Lin, "ShareRender: Bypassing GPU virtualization to enable fine-grained resource sharing for cloud gaming," in *Proceedings of the 25th ACM international conference on Multimedia*, ser. MM '17. Association for Computing Machinery, pp. 324–332, <https://github.com/CGCL-codes/ShareRender>. [Online]. Available: <https://dl.acm.org/doi/10.1145/3123266.3123306>
- [22] Z. Qi, J. Yao, C. Zhang, M. Yu, Z. Yang, and H. Guan, "VGRIS: Virtualized GPU resource isolation and scheduling in cloud gaming," vol. 11, no. 2, pp. 17:1–17:25. [Online]. Available: <https://dl.acm.org/doi/10.1145/2632216>
- [23] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, "My vm is lighter (and safer) than your container," in *Proceedings of the 26th Symposium on Operating Systems Principles*, ser. SOSP '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 218–233. [Online]. Available: <https://doi.org/10.1145/3132747.3132763>
- [24] M. Vrable, J. Ma, J. Chen, D. Moore, E. Vandekieft, A. C. Snoeren, G. M. Voelker, and S. Savage, "Scalability, fidelity, and containment in the potemkin virtual honeyfarm," *SIGOPS Oper. Syst. Rev.*, vol. 39, no. 5, p. 148–162, oct 2005. [Online]. Available: <https://doi.org/10.1145/1095809.1095825>
- [25] H. A. Lagar-Cavilla, J. A. Whitney, A. M. Scannell, P. Patchin, S. M. Rumble, E. De Lara, M. Brudno, and M. Satyanarayanan, "Snowflock: rapid virtual machine cloning for cloud computing," in *Proceedings of the 4th ACM European conference on Computer systems*, 2009, pp. 1–12.
- [26] L. Ao, G. Porter, and G. M. Voelker, "Faasnap: Faas made fast using snapshot-based vms," in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 730–746. [Online]. Available: <https://doi.org/10.1145/3492321.3524270>
- [27] R. W. Wisniewski, T. Inglett, P. Keppel, R. Murty, and R. Riesen, "mOS: an architecture for extreme-scale operating systems," in *Proceedings of the 4th International Workshop on Runtime and Operating Systems for Supercomputers*, ser. ROSS '14. Association for Computing Machinery, pp. 1–8. [Online]. Available: <https://dl.acm.org/doi/10.1145/2612262.2612263>
- [28] B. Bui, D. Mvondo, B. Teabe, K. Jioke, L. Wapet, A. Tchana, G. Thomas, D. Hagimont, G. Muller, and N. DePalma, "When eXtended para - virtualization (XPV) meets NUMA," in *Proceedings of the Fourteenth EuroSys Conference 2019*, ser. EuroSys '19. Association for Computing Machinery, pp. 1–15. [Online]. Available: <https://dl.acm.org/doi/10.1145/3302424.3303960>
- [29] NVIDIA, "Unlock next level performance with virtual gpu," <https://www.nvidia.com/en-us/data-center/virtual-solutions/>.
- [30] V. A. Sartakov, L. Vilanova, and P. Pietzuch, "Cubicleos: a library os with software componentisation for practical isolation," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 546–558. [Online]. Available: <https://doi.org/10.1145/3445814.3446731>
- [31] S. Kuenzer, V.-A. Băadoiu, H. Lefeuve, S. Santhanam, A. Jung, G. Gain, C. Soldani, C. Lupu, S. Teodorescu, C. Răducanu, C. Banu, L. Mathy, R. Deaconescu, C. Raiciu, and F. Huici, "Unikraft: fast, specialized unikernels the easy way," in *Proceedings of the Sixteenth European Conference on Computer Systems*, ser. EuroSys '21. Association for Computing Machinery, pp. 376–394. [Online]. Available: <https://dl.acm.org/doi/10.1145/3447786.3456248>
- [32] A. Madhavapeddy, R. Mortier, C. Rotsos, D. Scott, B. Singh, T. Gazagnaire, S. Smith, S. Hand, and J. Crowcroft, "Unikernels: library operating systems for the cloud," *SIGARCH Comput. Archit. News*, vol. 41, no. 1, p. 461–472, mar 2013. [Online]. Available: <https://doi.org/10.1145/2490301.2451167>
- [33] A. Raza, T. Unger, M. Boyd, E. B. Munson, P. Sohal, U. Drepper, R. Jones, D. B. De Oliveira, L. Woodman, R. Mancuso, J. Appavoo, and O. Krieger, "Unikernel linux (UKL)," in *Proceedings of the Eighteenth European Conference on Computer Systems*, ser. EuroSys '23. Association for Computing Machinery, pp. 590–605. [Online]. Available: <https://doi.org/10.1145/3552326.3587458>
- [34] F. Manco, C. Lupu, F. Schmidt, J. Mendes, S. Kuenzer, S. Sati, K. Yasukata, C. Raiciu, and F. Huici, "My VM is lighter (and safer) than your container," in *Proceedings of the 26th Symposium on Operating Systems Principles*, ser. SOSP '17.

- Association for Computing Machinery, pp. 218–233. [Online]. Available: <https://dl.acm.org/doi/10.1145/3132747.3132763>
- [35] N. Eiling, M. Kröning, J. Klimt, P. Fensch, S. Lankes, and A. Monti, “GPU acceleration in unikernels using cricket GPU virtualization,” in *Proceedings of the SC ’23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, ser. SC-W ’23. Association for Computing Machinery, pp. 1588–1595. [Online]. Available: <https://dl.acm.org/doi/10.1145/3624062.3624236>
- [36] “NVIDIA/open-gpu-kernel-modules,” original-date: 2022-05-02T22:00:04Z. [Online]. Available: <https://github.com/NVIDIA/open-gpu-kernel-modules>
- [37] P. Sharma, L. Chaufournier, P. Shenoy, and Y. C. Tay, “Containers and virtual machines at scale: A comparative study,” in *Proceedings of the 17th International Middleware Conference*, ser. Middleware ’16. Association for Computing Machinery, pp. 1–13. [Online]. Available: <https://doi.org/10.1145/2988336.2988337>
- [38] T. Kämäräinen, Y. Shan, M. Siekkinen, and A. Ylä-Jääski, “Virtual machines vs. containers in cloud gaming systems,” in *2015 International Workshop on Network and Systems Support for Games (NetGames)*, pp. 1–6, ISSN: 2156-8146. [Online]. Available: <https://ieeexplore.ieee.org/document/7382987>
- [39] “NVIDIA/nvidia-container-toolkit,” original-date: 2019-11-14T20:11:16Z. [Online]. Available: <https://github.com/NVIDIA/nvidia-container-toolkit>
- [40] SteamDB search. [Online]. Available: <https://steamdb.info/instantsearch/>
- [41] Valve Corporation. Steam search. [Online]. Available: <https://store.steampowered.com/search/?ndl=1>
- [42] T. Crider, “GloriousEggroll/wine-ge-custom,” original-date: 2019-11-21T06:03:28Z. [Online]. Available: <https://github.com/GloriousEggroll/wine-ge-custom>
- [43] B. Amstadt and M. K. Johnson, “Wine,” vol. 1994, no. 4, pp. 3–es.
- [44] “wine / wine · GitLab.” [Online]. Available: <https://gitlab.winehq.org/wine/wine>
- [45] “ValveSoftware/proton,” original-date: 2018-03-13T18:45:27Z. [Online]. Available: <https://github.com/ValveSoftware/Proton>
- [46] Buck DeFore. ProtonDB | gaming know-how from the linux and steam deck community. [Online]. Available: <https://www.protondb.com/>
- [47] A. Alvin. Window stations - win32 apps. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/winstation/window-stations>
- [48] A. Vinficius. GUI availability on windows container - issue #306 · microsoft/windows-containers. [Online]. Available: <https://github.com/microsoft/Windows-Containers/issues/306#issuecomment-1376440627>
- [49] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” vol. 2014, no. 239, p. 2:2.
- [50] P. Rebohle, “doitsujin/dxvk,” original-date: 2017-10-11T13:34:29Z. [Online]. Available: <https://github.com/doitsujin/dxvk>
- [51] J. Peters, “jp7677/dxvk-nvapi,” original-date: 2019-08-26T17:21:59Z. [Online]. Available: <https://github.com/jp7677/dxvk-nvapi>
- [52] “ValveSoftware/vkd3d,” original-date: 2019-10-31T23:50:07Z. [Online]. Available: <https://github.com/ValveSoftware/vkd3d>
- [53] H.-K. Arntzen, “HansKristian-work/vkd3d-proton,” original-date: 2019-09-19T10:37:39Z. [Online]. Available: <https://github.com/HansKristian-Work/vkd3d-proton>
- [54] flightlessmango, “flightlessmango/MangoHud,” original-date: 2020-01-28T03:18:11Z. [Online]. Available: <https://github.com/flightlessmango/MangoHud>
- [55] “GStreamer/gstreamer,” original-date: 2015-12-01T21:49:40Z. [Online]. Available: <https://gitlab.freedesktop.org/gstreamer/gstreamer>
- [56] “GameTechDev/PresentMon,” original-date: 2016-03-09T18:44:16Z. [Online]. Available: <https://github.com/GameTechDev/PresentMon>
- [57] NVIDIA Corporation. NVIDIA system management interface. [Online]. Available: <https://developer.nvidia.com/system-management-interface>
- [58] Advanced Micro Devices, Inc, “Open-source register reference for AMD family 17h processors models 00h-2fh,” RAPL page 139. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/56255_0SRR.pdf
- [59] R. Schone, T. Ilsche, M. Bielert, M. Velten, M. Schmidl, and D. Hackenberg, “Energy efficiency aspects of the AMD zen 2 architecture,” in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE Computer Society, pp. 562–571. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/Cluster48925.2021.00087>
- [60] Y. Li, C. Shan, R. Chen, X. Tang, W. Cai, S. Tang, X. Liu, G. Wang, X. Gong, and Y. Zhang, “GAgur: Quantifying performance interference of colocated games for improving resource utilization in cloud gaming,” in *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC ’19. Association for Computing Machinery, pp. 231–242. [Online]. Available: <https://doi.org/10.1145/3307681.3325409>
- [61] Q. Lu, J. Yao, H. Guan, and P. Gao, “gQoS: A QoS-oriented GPU virtualization with adaptive capacity sharing,” vol. 31, no. 4, pp. 843–855, conference Name: IEEE Transactions on Parallel and Distributed Systems.
- [62] K. Tian, Y. Dong, and D. Cowperthwaite, “A full GPU virtualization solution with mediated pass-through,” in *Proceedings of the 2014 USENIX conference on USENIX Annual Technical Conference*, ser. USENIX ATC’14. USENIX Association, pp. 121–132. [Online]. Available: <https://www.usenix.org/system/files/conference/atc14/atc14-paper-tian.pdf>
- [63] B. Wu, Z. Zhang, Z. Bai, X. Liu, and X. Jin, “Transparent [GPU] sharing in container clouds for deep learning workloads,” pp. 69–85, <https://github.com/pkusys/TGS>. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/wu>
- [64] J. Gu, S. Song, Y. Li, and H. Luo, “GaiaGPU: Sharing GPUs in container clouds,” in *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*, pp. 469–476. [Online]. Available: <https://ieeexplore.ieee.org/document/8672318>
- [65] F. Strati, X. Ma, and A. Klimovic, “Orion: Interference-aware, fine-grained GPU sharing for ML applications,” in *Proceedings of the Nineteenth European Conference on Computer Systems*, ser. EuroSys ’24. Association for Computing Machinery, pp. 1075–1092. [Online]. Available: <https://doi.org/10.1145/3627703.3629578>
- [66] AleksandarK. NVIDIA cracks down on CUDA translation layers, changes licensing terms. [Online]. Available: <https://www.techpowerup.com/319984/nvidia-cracks-down-on-cuda-translation-layers-changes-licensing-terms>
- [67] NVIDIA Corporation. License agreement for NVIDIA software development kits. EULA v12.6. [Online]. Available: <https://docs.nvidia.com/cuda/eula/index.html>
- [68] Y. Lee, VAR: Vulkan API Remoting for GPU-accelerated Rendering and Computation in Virtual Machines. Pennsylvania State University, google-Books-ID: jYzyzQEACAAJ.
- [69] C.-H. Hong, I. Spence, and D. S. Nikolopoulos, “GPU virtualization and scheduling methods: A comprehensive survey,” vol. 50, no. 3, pp. 35:1–35:37. [Online]. Available: <https://doi.org/10.1145/3068281>
- [70] “KhronosGroup/vulkan-loader,” original-date: 2018-05-03T19:59:26Z. [Online]. Available: <https://github.com/KhronosGroup/Vulkan-Loader>